

ICP133 - Fundamentos de Sistemas de Computação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
May 19, 2026

Aula 8 - Circuitos e Unidade Lógica e Aritmética

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
May 19, 2026

Circuito Somador

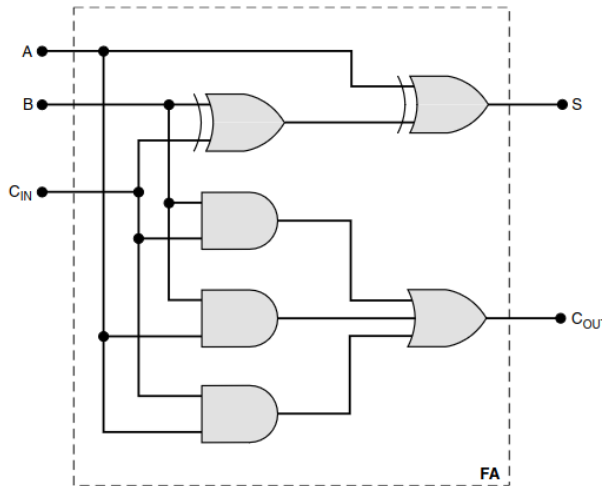


FIGURA 6.8 Circuito para um somador completo.

Circuito Somador - Tabela verdade

Entradas de bits da primeira parcela	Entradas de bits da segunda parcela	Entradas de bits do carry	Saída de bits da soma	Saída de bits do carry
A	B	C_{IN}	S	C_{OUT}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

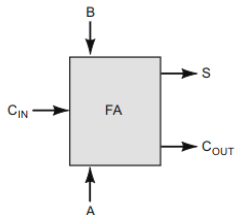


FIGURA 6.7 Tabela-verdade para um circuito somador completo.

Unidade Lógica e aritmética

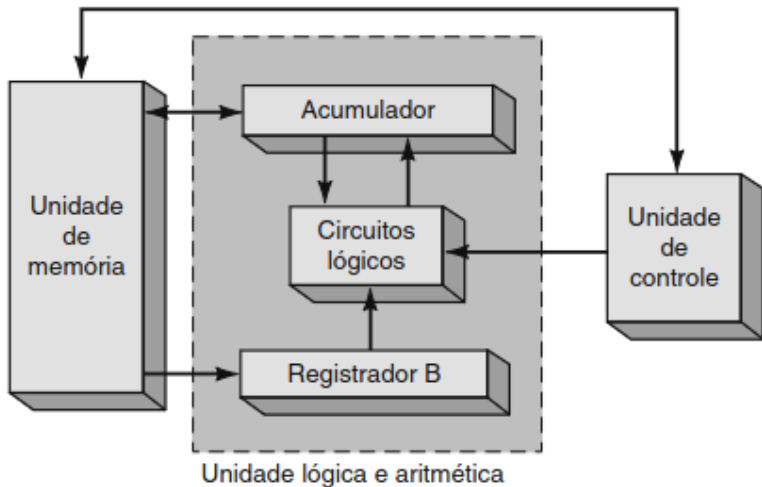


FIGURA 6.4 Blocos funcionais de uma ALU.

A Unidade Lógica e Aritmética (ULA / ALU)

- Todas as operações lógicas e aritméticas são realizadas na **unidade lógica e aritmética** (ALU) de um computador.
- **Objetivo principal:** Receber dados binários armazenados na memória e executar operações sobre eles, de acordo com instruções da unidade de controle.

Estrutura Básica

Contém pelo menos dois registradores (**Registrador B** e **Acumulador**) e uma **lógica combinacional** que realiza as operações sobre os números armazenados neles.

Fluxo Típico de Operações

1. **Instrução:** A unidade de controle decodifica a instrução indicando que um número de uma posição da memória deve ser somado ao **acumulador**.
2. **Carga:** O número a ser somado é transferido da memória para o **registrador B**.
3. **Processamento:** Os números de B e do acumulador são somados no circuito lógico. O resultado é enviado e **sobrescreve o acumulador**.
4. **Destino:** O novo valor no acumulador pode ser mantido para sucessivas somas ou transferido de volta para a memória.

O Papel do Acumulador

Ele "*acumula*" os resultados dos passos intermediários de problemas aritméticos que implicam múltiplos passos, além do resultado final.

As Quatro Operações na ALU

Apesar de a ALU ser baseada em circuitos somadores, ela consegue realizar todas as operações aritméticas fundamentais:

- **Soma:** Executada diretamente pelos circuitos somadores binários.
- **Subtração:** Realizada através da soma com o **complemento de dois** ($A - B = A + [B_{comp2}]$).
- **Multiplicação:** Implementada como uma sucessão de somas e deslocamentos de bits (*shifts*).
- **Divisão:** Processada através de subtrações sucessivas e comparações.

Divisão por Subtrações Sucessivas ($27 \div 5$)

Conceito: A ALU realiza a divisão subtraindo o divisor do dividendo repetidamente até que o resultado seja menor que o divisor.

- **Dividendo:** 27
- **Divisor:** 5

Processo Iterativo

1. $27 - 5 = 22$ (1ª subtração)
2. $22 - 5 = 17$ (2ª subtração)
3. $17 - 5 = 12$ (3ª subtração)
4. $12 - 5 = 7$ (4ª subtração)
5. $7 - 5 = 2$ (5ª subtração)

Resultado Final

- **Quociente:** 5
(Número de subtrações realizadas)
- **Resto:** 2
(Valor final menor que o divisor)

Lógica da ALU:

Contador de iterações =
Quociente
Resíduo final = Resto

A Máquina IAS

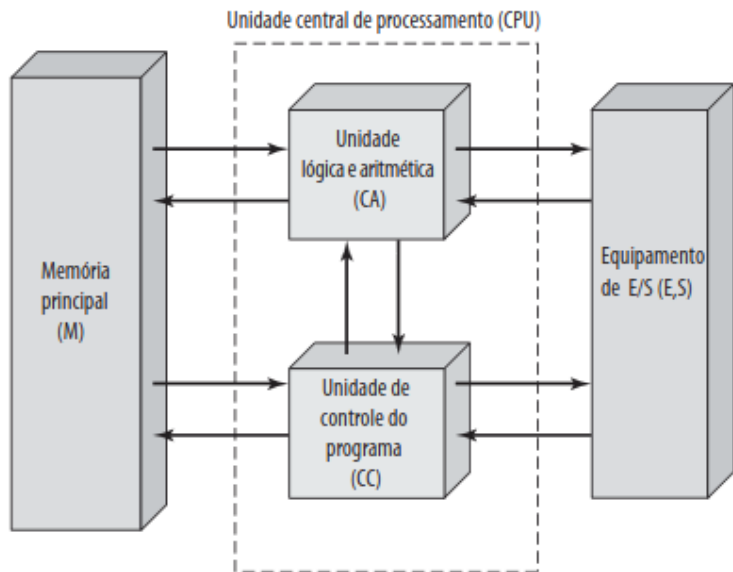
- **Origem:** Foi o primeiro computador eletrônico construído no *Institute for Advanced Study* (IAS) em Princeton, Nova Jersey.
- **A Máquina de von Neumann:** Recebe frequentemente este nome porque o artigo que descrevia o seu projeto foi editado por **John von Neumann**, professor de matemática da Universidade de Princeton e do IAS.
- **Linha do Tempo:** Construído sob sua direção, o projeto teve início em **1946** e foi concluído em **1951**.
- [Veja o link na Wikipédia](#)

Legado da Arquitetura

A organização geral desenvolvida passou a ser chamada de **Arquitetura de von Neumann**, embora tenha sido concebida e implementada também por outros pesquisadores do instituto.

Presente hoje na coleção do *Smithsonian National Museum of American History*.

Estrutura básica do Computador IAS

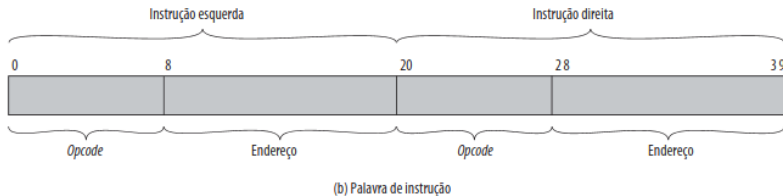
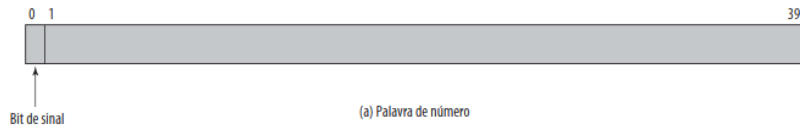


A Estrutura Geral do Computador IAS

O computador IAS foi baseado na proposta inicial de von Neumann e serve como base para os computadores modernos. Ele consiste em quatro componentes fundamentais:

- **Memória Principal (M):** Responsável por armazenar tanto os dados quanto as instruções do programa.
- **Unidade Lógica e Aritmética (CA - Central Arithmetic):** Unidade especializada para realizar as operações aritméticas elementares e lógicas.
- **Unidade de Controle (CC - Central Control):** Órgão central que decodifica as instruções e sequencia a execução das operações.
- **Entrada e Saída (I/O - Input/Output):** Unidades encarregadas de realizar o contato com o meio exterior de gravação (R).

Estrutura da memória



Formatos de Palavra no Computador IAS

A memória do IAS é organizada em 1.000 locais de armazenamento chamados **palavras** (*words*), onde cada palavra possui **40 bits**.

Representação de Dados

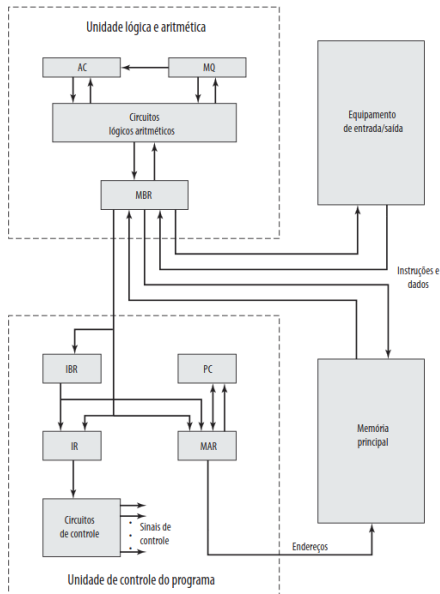
- **1 bit de sinal** seguido por um valor de **39 bits**.

Representação de Instruções

Uma palavra pode armazenar até **duas instruções de 20 bits** cada. Cada instrução é dividida em:

- **Código de Operação (Opcode):** 8 bits para especificar a operação.
- **Endereço:** 12 bits para apontar para uma das 1.000 palavras na memória.

Computador IAS mais a fundo



Registradores da Unidade de Controle e ULA

Para buscar e executar as instruções passo a passo, o IAS utiliza um conjunto específico de registradores internos:

- **mBR (Memory Buffer Register):** Contém a palavra que será enviada para a memória/E/S ou que acabou de ser recebida deles.
- **mAR (Memory Address Register):** Especifica o endereço de memória da palavra que será lida ou gravada via mBR.
- **IR (Instruction Register):** Guarda o opcode de 8 bits da instrução que está sendo executada no momento.
- **IBR (Instruction Buffer Register):** Armazena temporariamente a segunda instrução da palavra para ser executada a seguir.
- **PC (Program Counter):** Contém o endereço do próximo par de instruções que deve ser buscado na memória.

Registradores de Dados: AC e MQ

Para lidar com operandos e resultados da ALU, o IAS utiliza dois registradores especiais que trabalham em conjunto:

- **AC (Acumulador) e MQ (Multiplier Quotient):** Empregados para manter temporariamente os operandos e os resultados das operações.

Multiplicação de 40 bits → Resultado de 80 bits

Quando a ALU multiplica dois números de 40 bits, o resultado de 80 bits é dividido:

- **AC:** Armazena os 40 bits **mais significativos**.
- **MQ:** Armazena os 40 bits **menos significativos**.

O Ciclo de Instrução do IAS

O funcionamento do IAS baseia-se na repetição contínua de um ciclo de instrução, dividido em dois subciclos principais:

1. Ciclo de Busca (Fetch Cycle):

- O *opcode* da próxima instrução é carregado no **IR**.
- A parte do endereço é carregada no **MAR**.
- **Origem:** A instrução pode vir direto do **IBR** (se já estiver lá) ou ser buscada na memória via **MBR**.

2. Ciclo de Execução (Execute Cycle):

- Com o *opcode* no **IR**, o circuito de controle o interpreta.
- Sinais de controle são enviados para mover dados ou ativar funções da ALU.

Por que a Indireção de Registradores?

No IAS, apenas o **MAR** especifica endereços de leitura/escrita e apenas o **MBR** serve como origem/destino de dados da memória.

Justificativa de Hardware

- **Simplificação Eletrônica:** Reduz drasticamente a quantidade de caminhos de dados (*data paths*) e multiplexadores necessários.
- **Controle Centralizado:** Facilita o projeto dos circuitos lógicos da Unidade de Controle.

O Conjunto de Instruções do IAS (21 Instruções)

As 21 instruções do computador IAS podem ser agrupadas em 5 categorias funcionais:

- **Transferência de Dados:** Movem dados entre a memória e registradores da ALU, ou entre os próprios registradores da ALU.
- **Desvio Incondicional:** Altera a execução sequencial do programa, permitindo a criação de laços (*loops*).
- **Desvio Condicional:** Permite tomar decisões no código baseando-se em condições (ex: se o valor for positivo).
- **Aritméticas:** Operações matemáticas executadas diretamente pelos circuitos da ALU.
- **Modificação de Endereço:** Permite que endereços sejam calculados na ALU e inseridos em instruções na memória, garantindo alta flexibilidade de endereçamento.

O Conjunto de Instruções do IAS (21 Instruções)

Tabela 2.1 O conjunto de instruções do IAS

Tipo de instrução	Opcode	Representação simbólica	Descrição
Transferência de dados	00001010	LOAD MQ	Transfere o conteúdo de MQ para AC
	00001001	LOAD MQ,M(X)	Transfere o conteúdo do local de memória X para MQ
	00100001	STOR M(X)	Transfere o conteúdo de AC para o local de memória X
	00000001	LOAD M(X)	Transfere M(X) para o AC
	00000010	LOAD - M(X)	Transfere - M(X) para o AC
	00000011	LOAD M(X)	Transfere o valor absoluto de M(X) para o AC
	00000100	LOAD - M(X)	Transfere - M(X) para o acumulador
Desvio incondicional	00001101	JUMP M(X,0:19)	Apanha a próxima instrução da metade esquerda de M(X)
	00001110	JUMP M(X,20:39)	Apanha a próxima instrução da metade direita de M(X)
Desvio condicional	00001111	JUMP+ M(X,0:19)	Se o número no AC for não negativo, apanha a próxima instrução da metade esquerda de M(X)
	00010000	JUMP+ M(X,20:39)	Se o número no AC for não negativo, apanha a próxima instrução da metade direita de M(X)
Aritmética	00000101	ADD M(X)	Soma M(X) a AC; coloca o resultado em AC
	00000111	ADD M(X)	Soma M(X) a AC; coloca o resultado em AC
	00000110	SUB M(X)	Subtrai M(X) de AC; coloca o resultado em AC
	00001000	SUB M(X)	Subtrai M(X) de AC; coloca o resto em AC
	00001011	MUL M(X)	Multiplica M(X) por MQ; coloca os bits mais significativos do resultado em AC; coloca bits menos significativos em MQ
	00001100	DIV M(X)	Divide AC por M(X); coloca o quociente em MQ e o resto em AC
	00010100	LSH	Multiplica o AC por 2; ou seja, desloca à esquerda uma posição de bit
	00010101	RSH	Divide o AC por 2; ou seja, desloca uma posição à direita
Modificação de endereço	00010010	STOR M(X,8:19)	Substitui campo de endereço da esquerda em M(X) por 12 bits mais à direita de AC
	00010011	STOR M(X,28:39)	Substitui campo de endereço da direita em M(X) por 12 bits mais à direita de AC